Q. Explain all conditional statements with examples

Conditional statements in Python are used to make decisions based on conditions. They allow a program to execute different blocks of code depending on whether a condition is True or False.

For example, if a student scores 40 or more marks, the program displays "Pass". Otherwise, it displays "Fail".

Python has four main forms of conditional statements:

1. if statement
2. if-else statement
3. if-elif-else statement
4. Nested if statement

Let's understand each one step by step, with syntax, examples, and outputs.

1. The if statement

Definition: The if statement executes a block of code only when the given condition is True. If the condition is False, the block is skipped.

Syntax:

```
if condition:
  statement
```

Example:

```
age = 20
if age >= 18:
  print("You are eligible to vote")
```

Output:

You are eligible to vote

Explanation:

- age = 20 stores 20 in the variable age.
- age >= 18 checks whether the age is at least 18.
- The condition is True, so the print() statement executes.

If age = 15, the condition is False, and nothing is printed.

2. The if-else statement

Definition: The if-else statement executes one block of code when the condition is True and another block when the condition is False.

Syntax:

```
if condition:
    statement1
else:
  statement2
```

Example:

```
num = 7
if num % 2 == 0:
  print("Even number")
else:
  print("Odd number")
```

Output:

Odd number

Explanation:

- % is the modulus operator, which returns the remainder.
- 7 % 2 gives 1.
- Therefore, num % 2 == 0 is False.
- The else block executes and prints "Odd number".

If num = 8, the output will be Even number.

3. The if-elif-else statement

Definition: The if-elif-else statement is used when we need to check multiple conditions. Python checks the conditions from top to bottom and executes the first block whose condition is True. If none is True, the else block executes.

- if checks the first condition.
- elif means “else if” and checks another condition.
- else executes when all previous conditions are False.

Syntax:

```
if condition1:
    statement1
elif condition2:
    statement2
elif condition3:
    statement3
```

```
else:
    statement4
```

Example:

```
marks = 75
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
elif marks >= 50:
    print("Grade C")
else:
    print("Fail")
```

Output:

Grade B

Explanation:

1. marks >= 90 is False.
2. marks >= 75 is True.
3. Python prints "Grade B" and skips the remaining conditions.

Important: The order of conditions matters. Check the highest marks first in this example.

4. Nested if statement

Definition: A nested if statement means writing one if statement inside another if statement. It is used when a second condition should be checked only after the first condition is satisfied.

Syntax:

```
if condition1:
    if condition2:
        statement1
    else:
        statement2
else:
    statement3
```

Example:

```
age = 20
has_id = True

if age >= 18:
    if has_id:
        print("Entry allowed")
    else:
        print("ID required")
else:
    print("Entry not allowed")
```

Output:

Entry allowed

Explanation:

- First, Python checks whether age >= 18. It is True.
- Then, it checks has_id. This is also True.
- Therefore, it prints "Entry allowed".

If the person is 20 but has_id = False, the output will be "ID required".

5. Short-hand if statement

Python also allows us to write a simple if statement on a single line.

Example:

```
age = 20
if age >= 18: print("Eligible to vote")
```

Output:

Eligible to vote

This is useful for simple conditions, but using multiple lines is usually easier to read for beginners.

6. Short-hand if-else (Conditional Expression)

Definition: A conditional expression allows us to choose between two values in a single line based on a condition.

Syntax:

```
value_if_true if condition else value_if_false
```

Example:

```
age = 16

result = "Adult" if age >= 18 else "Minor"

print(result)
```

Output:

Minor

Explanation: Since age >= 18 is False, Python assigns "Minor" to result.

This is also called the ternary conditional expression.

7. Conditions using logical operators

Logical operators allow us to combine multiple conditions in an if statement.

Operator	Meaning	Example
and	Both conditions must be True	age >= 18 and age <= 60
or	At least one condition must be True	day == "Sat" or day == "Sun"
not	Reverses the result	not (age >= 18)

Example:

```
age = 25

if age >= 18 and age <= 60:
    print("Eligible")
else:
    print("Not eligible")
```

Output:

Eligible

Both conditions are True, so Python prints "Eligible".

8. Important rules for conditional statements

- Colon (:): Write a colon after the condition.
- Indentation: Indent the statements inside the conditional block, usually by four spaces.
- Comparison operators: Use == to compare two values. A single = assigns a value to a variable.
- Boolean values: A condition evaluates to True or False.
- Order of execution: In an if-elif-else structure, only the first matching block executes.